

Python Dictionary and Tuple Functions

Explorin Python Dictionary

Introduction

- Dictionary: A collection of key-value pairs.

Accessing Values in a Dictionary

○ Example:

```
my_dict = {'name': 'Alice', 'age': 25}
print(my_dict['name'])
print(my_dict.get('ages', "Unknown Key"))
```

Alice

Unknown Key

Updating/Adding Dictionary

- Example:

```
my_dict = {'name': 'Alice', 'age': 25}
my_dict['age'] = 26
my_dict['city'] = 'New York'
print(my_dict)
```

```
{'name': 'Alice', 'age': 26, 'city': 'New York'}
```

Deleting Dictionary Elements

○ Example:

```
my_dict = {'name': 'Alice', 'age': 25, 'city': 'New York'}  
del my_dict['city']  
print(my_dict)
```

```
{'name': 'Alice', 'age': 25}
```

Properties of Dictionary Keys

- Keys must be unique.
- Keys must be immutable (e.g., strings, numbers, or tuples).

Example:

```
my_dict = {(231231): 'This is value'} # Valid
# my_dict = {[1, 2]: 'value'} # Invalid, lists are mutable.
print(my_dict[231231])
```

```
This is value
```

Nested Dictionaries

```
nested_dict = {  
    "person1": {  
        "name": "Alice",  
        "age": 30,  
        "city": "New York"  
    },  
    "person2": {  
        "name": "Bob",  
        "age": 25,  
        "city": "Los Angeles"  
    }  
}  
  
print(nested_dict["person1"]["name"])
```

Built-in Dictionary Functions and Methods

- `len(dict)`: Returns the number of key-value pairs.

Example:

```
my_dict = {  
    'name': 'Alice',  
    'age': 25,  
    'city': 'New York',  
    'is_student': False  
}  
  
num_pairs = len(my_dict)  
print(num_pairs)
```

4

- `dict.keys()`: Returns all keys.

Example:

```
my_dict = {  
    'name': 'Alice',  
    'age': 25,  
    'city': 'New York',  
    'is_student': False  
}  
  
keys = my_dict.keys()  
print("The keys in the dictionary are:", keys)
```

The keys in the dictionary are: dict_keys(['name', 'age', 'city', 'is_student'])

dict.values(): Returns all values.

Example:

```
my_dict = {  
    'name': 'Alice',  
    'age': 25,  
    'city': 'New York',  
    'is_student': False  
}  
  
values = my_dict.values()  
print("The values in the dictionary are:", values)
```

```
The values in the dictionary are: dict_values(['Alice', 25, 'New York', False])
```

- `dict.items()`: Returns key-value pairs.

Example:

```
my_dict = {  
    'name': 'Alice',  
    'age': 25,  
    'city': 'New York',  
    'is_student': False  
}  
  
items = my_dict.items()  
print("The key-value pairs in the dictionary are:", items)
```

```
The key-value pairs in the dictionary are: dict_items([('name', 'Alice'), ('age', 25),  
('city', 'New York'), ('is_student', False)])
```

1st

- Create a dictionary named students where the keys are student names (strings) and the values are their grades (integers).
- Implement the following features in your program:
 1. Add a new student and grade to the dictionary.
 2. Update the grade of an existing student.
 3. Delete a student from the dictionary.
 4. Display all students and their grades in a readable format.
 5. Exit the program (yes/no)

Use a loop to provide the user with a menu of options to perform these actions repeatedly until they choose to exit.

Time to Practice:

Scenario: You are tasked to create a program for a small community grocery store. The store wants to maintain a simple inventory system where they can:

1. List available items.
2. Add new items with their quantities and prices.
3. Update existing stock levels.
4. Update existing Price
5. Give options to user to continue choosing option
6. Exit the program when all updates are complete.